

# Secure Web Development — Testing & Demo Guide

## Lab Reservation System (CCAPDEV Phase 3 — Group 10)

This guide maps every line item on DLSU's **Secure Web Development Case Project Checklist (Rev. 02-2026, /58)** to a concrete, repeatable test against this specific application, so the group can practice independently and run a confident, well-paced demo for the professor.

| Role (Checklist term) | Role (App term) | Username   | Password    |
|-----------------------|-----------------|------------|-------------|
| Website Administrator | Administrator   | admin      | Admin123!   |
| Product Manager       | Lab Manager     | labmanager | Manager123! |
| Customer              | Student         | student    | Student123! |

These accounts are created by `node init_db.js`. Run it before every practice session and before the actual demo so lockouts / password-age flags from prior tests are reset to a clean state.

## 0. One-Time Setup

1. Make sure MongoDB is running locally (default: `mongodb://127.0.0.1:27017/userdb`).
2. `npm install`
3. `node init_db.js` — (re)creates the three demo accounts and seed data. Safe to re-run before every test session to reset lockouts/state.
4. `npm start`, then open `http://localhost:3000`.

### Two testing modes you'll use throughout:

1. **Browser (UI) testing** — proves the feature works for a normal user and looks right on screen.
2. **Direct API testing (curl/Postman)** — proves validation happens on the *server*, not just in the browser's JavaScript. This matters because a checklist grader may bypass your HTML form entirely. Example base request:

```
curl -i -X POST http://localhost:3000/api/login \  
  -H "Content-Type: application/json" \  
  -d '{"username":"student","password":"wrongpass"}'
```

For endpoints that require a session (most of them), first log in with curl using `-c cookies.txt` to save the session cookie, then reuse it with `-b cookies.txt` on the next request.

## Table of Contents

- 1.1 Accounts (3 roles)
- 2.1 Authentication (13 items)

- 2.2 Authorization / Access Control (3 items)
- 2.3 Data Validation (3 items)
- 2.4 Error Handling & Logging (7 items)
- Suggested Demo Script & Running Order
- Self-Grading Scorecard (mirrors the official /58 sheet)

## 1.0 Pre-Demo Requirements

---

### 1.1 Accounts — at least 1 per user type

#### HOW TO TEST

1. Log in as `admin` / `Admin123!` — should land on the Admin Dashboard ( `/admin` ).
2. Log in as `labmanager` / `Manager123!` — should land on the Lab Manager Dashboard ( `/manager` ).
3. Log in as `student` / `Student123!` — should land on the homepage ( `/home` ) with reservation access only.

#### PASS CRITERIA

All three accounts exist, log in successfully, and are routed to role-appropriate views.

## 2.1 Authentication

### 2.1.1 Require authentication for all pages/resources except public ones

#### WHAT'S PUBLIC IN THIS APP

`/login`, `/register`, `/forgot-password`, and static assets under `/css`, `/js`, `/images` (these must stay public — the login/register pages need them to render). Everything else requires a session.

#### HOW TO TEST

1. Log out (or open a private/incognito window).
2. Try to browse directly to `/home`, `/account-profile`, `/reservation`, `/admin`, `/manager`.
3. Also try the JSON API directly: `curl -i http://localhost:3000/api/user`.

#### EXPECTED RESULT

Browser routes redirect to `/login`; API calls return `401 {"success":false,"message":"Please login first"}`. Login/register/forgot-password pages and static assets load without a session.

### 2.1.2 Authentication controls fail securely

#### HOW TO TEST

- In DevTools → Application → Cookies, edit/corrupt the `lrs.sid` session cookie value, then reload a protected page.
- Delete the cookie entirely and reload a protected page.

#### EXPECTED RESULT

Access is denied (redirect to login / 401) in every tampered case — the default is *deny*, never silently logged-in or granted access.

### 2.1.3 Only strong, salted, one-way password hashes are stored

#### HOW TO TEST

1. Open a Mongo shell: `mongosh mongodb://127.0.0.1:27017/userdb`
2. Run: `db.users.find({}, {username:1, password:1}).pretty()`

#### EXPECTED RESULT

Every `password` value looks like `$2a$10$...` (bcrypt, 10 salt rounds) — never plaintext. Point out during the demo that this is `bcrypt.hash(password, 10)` in `controllers/authController.js`.

#### 2.1.4 Generic authentication failure message

##### HOW TO TEST

1. Try logging in with a username that doesn't exist.
2. Try logging in with `student` and a wrong password.

##### EXPECTED RESULT

Both cases show the exact same message: *"Invalid username and/or password."* Never "user not found" vs "wrong password."

#### 2.1.5 / 2.1.6 Password complexity & length requirements

##### POLICY IN THIS APP

At least 8 characters (max 72), with at least one uppercase letter, one lowercase letter, one digit, and one special character.

##### HOW TO TEST

- Go to Register and try weak passwords: `abc`, `alllowercase1!`, `ALLUPPER1!`, `NoSpecialChar1`.
- Then use a compliant password, e.g. `Test1234!`.

##### EXPECTED RESULT

Weak attempts are rejected with the policy message; the compliant password succeeds.

#### 2.1.7 Password entry is obscured on screen

##### HOW TO TEST

On Login, Register, and Account Profile (change password), confirm the password fields render as dots and the browser DevTools element inspector shows `<input type="password">`.

### 2.1.8 Account lockout after repeated invalid attempts

#### POLICY IN THIS APP

5 consecutive failed logins → account locked for **10 minutes** (even a correct password is rejected while locked).

#### HOW TO TEST

1. Using the `student` account, submit the wrong password **5 times in a row**.
2. On the 6th attempt, use the *correct* password.

#### EXPECTED RESULT

After the 5th failure the account locks; the 6th attempt (even correct) returns `423 "Account is temporarily locked. Please try again later."`

**Demo pacing tip:** do this test *first*, before anything else that needs the student account, so the 10-minute lock has time to expire (or simply re-run `node init_db.js` afterward to reset it, or clear it manually in Mongo: `db.users.updateOne({username:"student"},{$set:{lockUntil:null,failedLoginAttempts:0}})` ). You do not need to wait out the full 10 minutes on demo day — showing the 6th attempt getting blocked is sufficient proof.

### 2.1.9 Password-reset questions support sufficiently random answers

#### HOW THIS APP SATISFIES IT

Instead of picking from a canned low-entropy list ("favorite color", "pet's name"), each user **writes their own** security question and answer at registration (question: 10–120 chars, answer: 6–80 chars).

#### HOW TO TEST

1. Register a new account and set a specific custom question, e.g. "What was the callsign of my first drone build?"
2. Go to Forgot Password, enter that username — confirm your custom question is shown back (not a generic one).
3. Answer correctly and set a new compliant password.

**2.1.10** Prevent password re-use**HOW TO TEST (NEEDS 2 PASSWORD CHANGES 1+ DAY APART, OR USE THE RESET FLOW WHICH IS EXEMPT FROM THE AGE RULE)**

1. Via Forgot Password, reset the `student` password to `Reused123!` .
2. Immediately reset it again (answer the security question again) to a different password, e.g. `Second456@` .
3. Reset a third time attempting to reuse `Reused123!` or `Second456@` .

**EXPECTED RESULT**

The third reset is rejected: *"New password cannot reuse an old password."* The app keeps a history of the last 5 password hashes per user to check against.

**2.1.11** Passwords must be  $\geq 1$  day old before they can be changed again**HOW TO TEST**

1. Log in, go to Account Profile, change the password (enter current password + a new compliant one).
2. Immediately try to change the password again.

**EXPECTED RESULT**

The second attempt is rejected: *"Password must be at least one day old before it can be changed again."*

This is the easiest way to *prove* the control on demo day — showing the immediate rejection is the evidence, you don't need to wait 24 hours. (For your own practice only, you can backdate it in Mongo to test the success path: `db.users.updateOne({username:"student"},{$set:{passwordChangedAt:new Date(Date.now()-2*86400000)}})` .)

Note: this rule intentionally does not apply to the Forgot Password flow, since that exists to recover a locked-out account and shouldn't be blocked by its own age rule.

**2.1.12** Report last account use at next successful login**HOW TO TEST**

1. Fail a login once on purpose (wrong password), then log in successfully right after.
2. Observe the homepage.

**EXPECTED RESULT**

A banner appears near the top of the homepage, e.g. *"Last login attempt failed on [timestamp]."* On a clean successful login streak it instead shows *"Last successful login was on [timestamp]."*

**2.1.13** Re-authenticate before critical operations (password change)**HOW TO TEST**

- Go to Account Profile → try to set a new password while leaving "Current Password" blank.
- Then try again with the wrong current password.
- Then try again with the correct current password.

**EXPECTED RESULT**

Blank → rejected ("Current password is required"). Wrong → 403 "Current password is incorrect." Correct → proceeds (subject to the 1-day rule and reuse check above).



## 2.2 Authorization / Access Control

### 2.2.1 Single site-wide component checks access authorization

#### WHERE IT LIVES

`middleware/authMiddleware.js` exports `requireLogin` and `requireRole(...roles)`, which every protected route in `routes/*.js` is wired through — there is no duplicated/ad-hoc auth logic per route.

#### HOW TO DEMONSTRATE

Open `routes/pageRoutes.js`, `routes/apiRoutes.js`, `routes/reservationRoutes.js`, and `routes/authRoutes.js` side by side and point out every sensitive route passes through `authMiddleware.requireLogin` or `authMiddleware.requireRole(...)` before its controller runs.

### 2.2.2 Access controls fail securely

#### HOW TO TEST

1. Log in as `student`, then browse directly to `/admin` and `/manager`.
2. Log in as `labmanager`, then browse directly to `/admin`.
3. As `student`, call `curl -b cookies.txt http://localhost:3000/api/admin/logs`.

#### EXPECTED RESULT

Student is denied both `/admin` and `/manager` (custom "Access Denied" page, HTTP 403). Lab Manager is denied `/admin` (admin-only). The logs API returns `403 {"success":false,"message":"Access denied"}` for non-admins. Default is always deny, never "allow and hide the button."

### 2.2.3 Enforce application logic to comply with business rules

#### HOW TO TEST — RESERVATION OWNERSHIP (IDOR CHECK)

1. As `student`, create a reservation.
2. Log in as a second student account, and try to delete/update the first student's reservation directly via the API (same lab/seat/date/time or reservation id).

Rejected with `403 "You can only modify/remove your own reservations."` (Lab Manager/Admin are allowed to manage any reservation.)

#### HOW TO TEST — RESERVATION INPUT RULES

- Submit a reservation for a lab name not in the whitelist (only `G404A`, `G404B`, `V101` are valid) → rejected.
- Submit a date more than 7 days out, or a Sunday → rejected (labs aren't bookable on Sundays or far in advance).
- Submit a non-adjacent start/end time pair (e.g. 7:30–9:00 instead of 7:30–8:00) → rejected (must be a single valid 30-min slot).
- Try booking the same lab/seat/date/time twice → second attempt returns `409 "This seat is already reserved for that time."`
- As a student, try to submit a reservation with a `walkInName` (walk-in bookings are a Lab-Manager-only feature) → rejected with `403 "Only Lab Managers can create walk-in reservations."`

## 2.3 Data Validation

### 2.3.1 Validation failures result in rejection — no sanitizing

#### HOW TO TEST

Send a NoSQL-injection-style payload directly to the API (bypassing the browser form), e.g.:

```
curl -i -X POST http://localhost:3000/api/login \  
-H "Content-Type: application/json" \  
-d '{"username": {"$ne": null}, "password": {"$ne": null}}'
```

#### EXPECTED RESULT

The request is **rejected outright** (400/401 generic error) — the app does not try to "clean up" or coerce the malicious object into a safe string and continue; any value containing `$`, `{`, or `}` fails the type/format check in `cleanString()` and the whole request is refused.

### 2.3.2 Validate data range

#### HOW TO TEST

- Reservation date more than 7 days in the future → rejected.
- Profile bio/description over 300 characters → rejected.
- Password shorter than 8 or longer than 72 characters → rejected.

### 2.3.3 Validate data length

#### HOW TO TEST

- Username outside 3–30 characters, or containing symbols/spaces → rejected (must be letters/numbers/underscore only).
- Security question shorter than 10 or longer than 120 characters → rejected.
- Security answer shorter than 6 or longer than 80 characters → rejected.
- Email longer than 100 characters → rejected.

## 2.4 Error Handling and Logging

---

### 2.4.1 Error handlers don't leak debugging/stack-trace info

#### HOW TO TEST

Trigger a server-side error (e.g. hit an endpoint with a malformed request that causes an exception, or stop MongoDB briefly and try to log in) and inspect the HTTP response body/rendered page.

#### EXPECTED RESULT

The client only ever sees a generic message like *"Something went wrong. Please try again later."* The real stack trace is only ever printed server-side via `console.error(err)`, never sent to the browser.

### 2.4.2 Generic error messages and custom error pages

#### HOW TO TEST

- Visit a non-existent URL, e.g. `/this-page-does-not-exist` → should render a custom "Page Not Found" page, not the default Express/stack-trace page.
- Trigger a 403 (student visiting `/admin`) → custom "Access Denied" page.

### 2.4.3–2.4.7 Logging controls

#### WHERE IT LIVES

Every security-relevant action calls `writeLog(req, eventType, status, details)` (`utils/security.js`), which stores an `AuditLog` document with event type, success/failure, username, role, IP, method, and path.

#### HOW TO TEST — SUCCESS + FAILURE BOTH LOGGED (2.4.3)

Log in successfully once, then fail a login once. Both should appear as separate `LOGIN` entries with different `status` values.

#### HOW TO TEST — LOGS RESTRICTED TO ADMINS ONLY (2.4.4)

- As `admin`, open `/admin` — the dashboard's log table should be visible.
- As `student` or `labmanager`, try `curl -b cookies.txt http://localhost:3000/api/admin/logs` → `403 Access denied`.

#### HOW TO TEST — VALIDATION FAILURES LOGGED (2.4.5)

Submit an invalid registration (weak password) and then check the admin log table for a `VALIDATION / failure` entry.

#### HOW TO TEST — AUTH ATTEMPTS LOGGED (2.4.6)

Check the log table after both a successful and a failed login for `LOGIN` entries (and `ACCOUNT_LOCKOUT` if you triggered 2.1.8).

#### HOW TO TEST — ACCESS-CONTROL FAILURES LOGGED (2.4.7)

After trying to visit `/admin` as a student (2.2.2), check the log table for an `ACCESS_CONTROL / failure` entry noting the insufficient role.

## Suggested Demo Script & Running Order

---

A live demo is time-boxed, so sequence things to avoid dead air (waiting on the 10-minute lockout or the 1-day password rule) and to reuse the same login session where possible. Suggested order:

1. **Kick off the lockout test first** (2.1.8) on a throwaway/test account — while the professor watches the 6th attempt fail, narrate the other authentication controls (hashing, generic messages) using the admin's Mongo view in a second window.
2. Show **account roles & login** for all three demo users (1.1), pointing out different landing dashboards.
3. Show **registration validation** live: weak password, short username, mismatched confirm-password, then a successful registration with a custom security question (2.1.5/2.1.6/2.1.9/2.3.x).
4. Show **generic failure message** (2.1.4) with two quick failed logins (bad username, then bad password).
5. Show **authorization boundaries** (2.2.2): student hitting `/admin` and `/manager` directly by URL.
6. Show **re-authentication + password reuse + 1-day rule** together in the Account Profile flow (2.1.13, 2.1.10, 2.1.11) — the rejections themselves are the proof, no waiting required.
7. Show **reservation business rules** (2.2.3): try an out-of-range date, then a double-booking, then (as a second student) try to edit someone else's reservation.
8. Show the **NoSQL-injection-style curl request** being rejected (2.3.1) — this is a strong, memorable moment for a security-focused grader.
9. Finish on the **Admin Dashboard**: show the audit log table, filter by event type/status, and point out that only the admin account can reach this page and its API (2.4.3–2.4.7), and show a 404 and a 500-style generic error page (2.4.1, 2.4.2).

**Before the actual demo:** run `node init_db.js` to reset all accounts to a clean, unlocked state with fresh password-age timestamps, so none of your practice runs leave the real demo accounts locked out or password-change-blocked.

## Self-Grading Scorecard

Use this to mock-grade yourselves before the real demo. Mirrors the official checklist's Complete(2)/Incomplete(1)/Missing(0) scale, max 58.

| #           | Requirement                                         | Verified? |
|-------------|-----------------------------------------------------|-----------|
| 1.1.1–1.1.3 | Admin, Lab Manager, Student accounts exist and work |           |
| 2.1.1       | All non-public pages/resources require auth         |           |
| 2.1.2       | Auth controls fail securely (default deny)          |           |
| 2.1.3       | Passwords stored as bcrypt salted hashes only       |           |
| 2.1.4       | Generic "Invalid username and/or password"          |           |
| 2.1.5       | Password complexity enforced                        |           |
| 2.1.6       | Password length enforced (8–72)                     |           |
| 2.1.7       | Password entry obscured (dots)                      |           |
| 2.1.8       | Lockout after 5 failed attempts, 10-min lock        |           |
| 2.1.9       | Reset question supports high-entropy answers        |           |
| 2.1.10      | Password reuse prevented                            |           |
| 2.1.11      | 1-day minimum age before password change            |           |
| 2.1.12      | Last login use reported at next login               |           |
| 2.1.13      | Re-auth required before password change             |           |
| 2.2.1       | Single shared access-control component              |           |
| 2.2.2       | Access controls fail securely                       |           |
| 2.2.3       | Business rules enforced (ownership, booking rules)  |           |
| 2.3.1       | Invalid input rejected, not sanitized               |           |
| 2.3.2       | Data range validated                                |           |
| 2.3.3       | Data length validated                               |           |
| 2.4.1       | No stack traces / debug info shown to users         |           |
| 2.4.2       | Generic messages + custom error pages (404/500/403) |           |
| 2.4.3       | Logging covers success and failure events           |           |
| 2.4.4       | Logs restricted to admin only                       |           |
| 2.4.5       | Input validation failures logged                    |           |
| 2.4.6       | Authentication attempts logged                      |           |
| 2.4.7       | Access control failures logged                      |           |

Prepared for internal group practice — based on Machine Project Checklist - Web App.pdf (DLSU CCS, Rev. 02-2026) and the current state of this repository's code.